



SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

MINI PROJECT:

Vaccine Queue Simulation System for Elderly

GROUP 6

MEMBER:

CHEW CHIU XIAN (A23CS0061)

HARINI A/P SANGARAN (A23CS0081)

MOHAMED ALI MOHAMED ALI (A21EC0287)

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

LECTURER:

DR. ZURAINI BINTI ALI SHAH

Table of Content

PART 1: INTRODUCTION

1.1 Synopsis	3
1.2 Objective	4

PART 2: SYSTEM ANALYSIS AND DESIGN (USE CASE AND CLASS DIAGRAM)

2.1 System Requirements	5
2.2 Class Diagram	6
2.3 Use Case Diagram	7

PART 3: SYSTEM PROTOTYPE

3.1 Code	8
3.2 Output of the Code	12

PART 4: Task Distribution and Log Sheet

4.1 Task Distribution Table	16
4.2 Group Meeting Log Sheet	17

Mini Project System Demo Link	17
--------------------------------------	-----------

Part 1: Introduction

1.1 Synopsis

In this mini project, we are building a system called the Vaccine Queue Simulation System, which is designed to make the vaccination process smooth, fair, and efficient for elderly individuals. Elderly people are often given priority in vaccination programs, so it's essential to create a system that organizes their turn in an orderly way.

This system follows the First-In-First-Out (FIFO) principle, which means the first person to join the queue will be the first one to receive the vaccine. This method ensures fairness and avoids confusion or delays.

The project focuses on using the Queue data structure to manage the process. By applying this concept, we can simulate how queues work in real life and demonstrate how data structures are used to solve practical problems. For example, when someone arrives, they are added to the end of the queue, and when it's their turn, they are removed from the front of the queue. This system ensures everyone gets vaccinated in the correct order without skipping or missing anyone.

1.2 Objective

The main objective of this project is to design and develop a simulation system using the **Queue data structure** in C++. This system will mimic the operations of a real-life vaccination queue, where elderly individuals arrive, wait for their turn, and are served one by one in the correct order.

To achieve this, the system will include the following features:

1. **Adding individuals to the queue:** When a new person arrives, their details will be added to the queue at the back, just like they would wait in line at a vaccination center.
2. **Removing individuals from the queue:** After a person has been vaccinated, they will be removed from the front of the queue, allowing the next person in line to take their turn.
3. **Displaying the current queue:** The system will show the current status of the queue, including the people waiting and their order. This feature ensures transparency and helps manage the vaccination process smoothly.
4. **Handling special cases:** The system will handle scenarios like an empty queue (when there are no individuals waiting) or a full queue (when the system has reached its limit of people).

By implementing these features, we aim to:

- Simplify the management of vaccine queues for elderly individuals.
- Demonstrate how the **Queue data structure** can be applied in real-world scenarios.
- Ensure fairness and order in the vaccination process through systematic queue management.
- Handling overflow and underflow avoids crashes or bugs in the system, ensuring a smooth operation at all times

This project not only helps in understanding the practical application of queues but also highlights their importance in designing systems that deal with order, time, and efficiency.

Part 2: System requirement, analysis & design

2.1 System Requirements

1. Add a person to the queue (enqueue)

- In the queue data structure, a person's details are stored in **an array or linked list**.
- The rear pointer keeps track of where the last person is in the queue.
- When a person is added, the system checks if the queue is full (overflow condition). If not, the new person's details are added at the position pointed to by the rear, and the rear pointer is updated.

2. Remove a person after vaccination (dequeue)

- The **front** pointer keeps track of the person at the front of the queue.
- When a person is removed, their details are retrieved from the position pointed to by the **front**. The **front** pointer is then moved to the next position.
- If the **front** and **rear** pointers are the same, the queue becomes empty, and both pointers are reset to -1 (indicating an empty queue).

3. Display the current queue

- The system iterates through the queue, starting from the **front** pointer and going up to the **rear** pointer.
- It prints the details of each person in the queue, showing the order they will be vaccinated.

4. Handle edge cases such as queue overflow and underflow

- Edge cases are special scenarios that the system needs to handle gracefully to avoid errors.
- There are two types of edge cases:

a) Overflow (Queue is Full):

- the queue reached maximum capacity, and no more individuals can be added.
- The system checks if the **rear** pointer plus one equals the **front** pointer (circular queue). If yes, the queue is full, and a message that says that queue is overflowed is displayed

b) Underflow (Queue is Empty):

- occurs when there are no individuals in the queue.
- Before removing a person (dequeue), the system checks if the **front** pointer is -1. If yes, the queue is empty, and a message that states that the queue is underflowed is displayed.

2.2 CLASS DIAGRAM

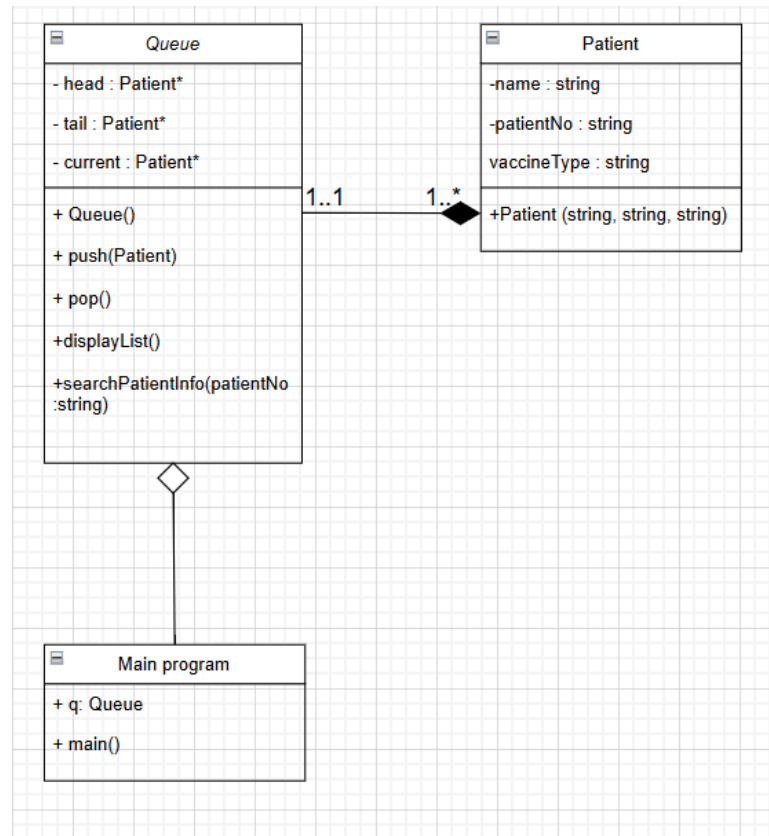
The class diagram shows the structure and relationships between the Patient, Queue, and Main Program classes in the Vaccine Queue Simulation System.

Patient Class: Represents individual patients with attributes like name, patientNo and vaccineType, linked using the next pointer.

Queue Class: Manages the patient queue with methods for adding, removing, displaying and searching patients.

Main Program: Provides a menu-driven interface to interact with the queue.

The Queue composes Patient objects, while the Main Program aggregates the Queue for operations.



2.3 USE CASE DIAGRAM

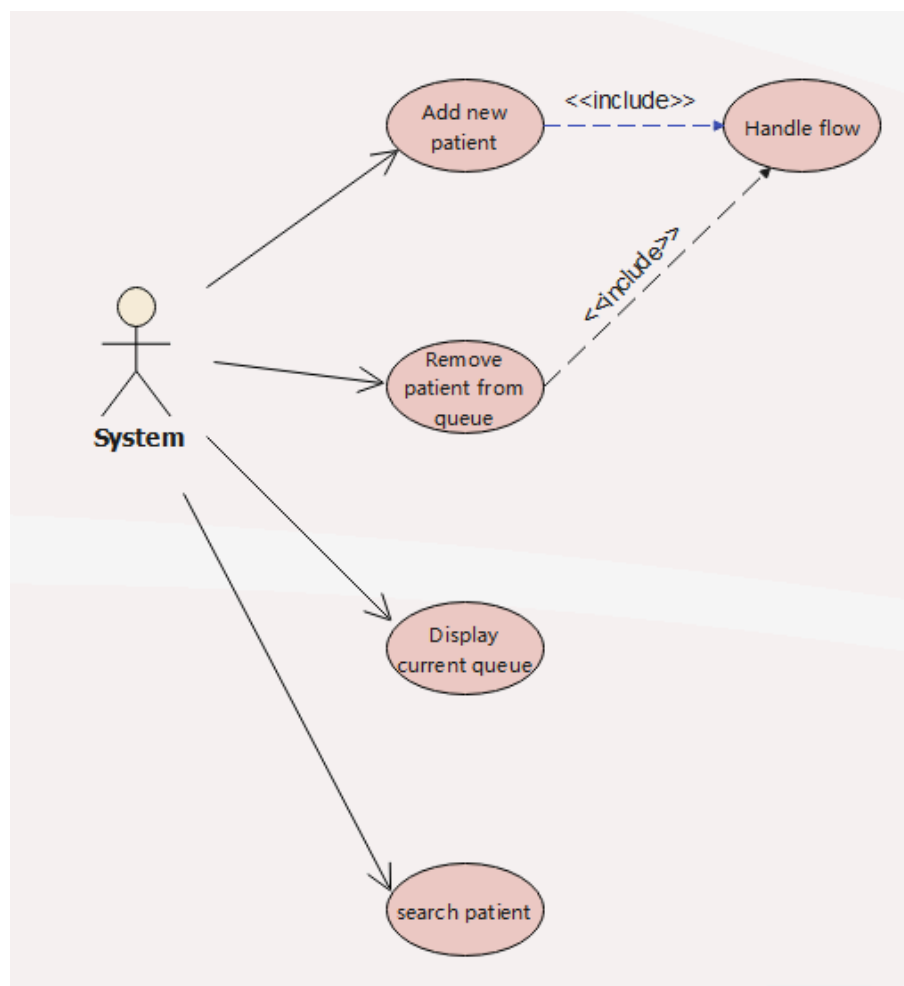
Figure 2.3 highlights system functionality and interactions between actors and use cases.

Actor:

- **System:** Manages the queue by adding, removing, displaying and searching for patients.

Use Cases: Include adding/removing patients, displaying the queue, searching patient details and handling edge cases (overflow/underflow).

This diagram emphasizes the System interactions with the patient and the supporting functionality.



Part 3: Development Activities and Coding in C++

3.1 Code :

```
#include <iostream>
#include <string>
// Group 6 : Vaccine Queue System for Elderly
using namespace std;

class Patient {
public:
    string name, patientNo, vaccineType;
    Patient* next;
    Patient* current;

    Patient(string name, string patientNo, string vaccineType) { //Constructor
        this->name = name;
        this->patientNo = patientNo;
        this->vaccineType = vaccineType;
        this->next = nullptr;
    }
};

class Queue {
private:
    Patient* head;
    Patient* tail;
    Patient* current;

public:
    Queue() : head(nullptr), tail(nullptr) {} //Constructor

    // Task 1: Insert new patient in the list (Chew)
    void push(Patient p) {
        Patient* newNode = new Patient(p);
        if (head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            tail = tail->next;
        }
    }

    // Task 2: Delete the patient who has received the vaccine (Mohamad Ali)
    void pop() {
```

```

if (head == nullptr) {
    cout << "No patients in the list." << endl << endl;
    return;
} else {
    if (head == tail) {
        cout << "Patient name : " << head->name << endl;
        cout << "Patient Number : " << head->patientNo << endl << endl;
        delete head;
        head = tail = nullptr;
    } else {
        Patient* temp = head;
        head = head->next;
        cout << "Patient name : " << temp->name << endl;
        cout << "Patient Number : " << temp->patientNo << endl << endl;
        delete temp;
    }
}
}

```

// Task 3 : Display the list (Syarifah)

```

void displayList() {
    current = head;
    int index = 0;

    if (current == nullptr) { // If the list is empty, display this message
        cout << "No patients in the list." << endl << endl;
        return;
    }

    while (current != nullptr) {
        index++;
        cout << "Patient " << index << ": " << endl;
        cout << "Patient name : " << current->name << endl;
        cout << "Patient Number : " << current->patientNo << endl;
        cout << "Vaccine type to be received : " << current->vaccineType << endl;
        cout << endl;
        current = current->next;
    }
}

```

//Task 4 : Search Patient Details (Harini)

```

void searchPatientInfo (string patientNo) {
    current = head;
    while (current != nullptr && current->patientNo != patientNo) {

```

```

        current = current->next;
    }
    if (current != nullptr) {
        cout << "Patient name : " << current->name << endl;
        cout << "Patient Number : " << current->patientNo << endl;
        cout << "Vaccine type to be received : " << current->vaccineType << endl;
        cout << endl;
    } else {
        cout << "Patient details do not exist in the list." << endl << endl;
    }
}

};

int main() {
    int choice;
    Queue q;

    //Display program title
    cout << "====Vaccine Queue System for Elderly====" << endl;
    do {
        //Display menu
        cout << endl << "Menu :" << endl;
        cout << "1. Insert New Patient Details" << endl;
        cout << "2. Delete Patient Details Who Have Received Vaccine" << endl;
        cout << "3. Display All Records" << endl;
        cout << "4. Search Patient Details" << endl;
        cout << "5. Exit" << endl;
        cout << endl << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: {
                //Input patient info into the system
                string name, patientNo, vaccineType;

                cout << "Enter Patient Name : ";
                cin.ignore();
                getline(cin, name);

                cout << "Enter Patient Number: ";
                cin >> patientNo;

                cout << "Enter Vaccine Type: ";
                cin >> vaccineType;

                Patient p(name, patientNo, vaccineType);
                q.push(p);
            }
        }
    } while (choice != 5);
}

```

```

        break;
    }

    case 2: { //Delete patient info from system
        cout << "Delete Record for Patient Who Have Received Vaccine : ";
        cout << endl;
        q.pop();
        break;
    }

    case 3: //Display list of patients
        q.displayList();
        break;

    case 4: { //Search patient info
        string patientNo;

        cout << "Enter Patient Number : ";
        cin >> patientNo;

        q.searchPatientInfo(patientNo);
        break;
    }

    case 5: //Exit system
        cout << "Exiting program." << endl;
        break;

    default:
        cout << "Invalid choice! Please try again." << endl;
    }
} while (choice != 5);
}

```

3.2 Output of The Code:

Vaccine Queue System Menu

```
=====Vaccine Queue System for Elderly=====
Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit
```

There are 4 options that users can select from the menu, which are:

1. Inserting New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Task 1 : Inserting New Patient Details

```
=====Vaccine Queue System for Elderly=====

Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 1
Enter Patient Name : John
Enter Patient Number: P0001
Enter Vaccine Type: Pfizer

Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 1
Enter Patient Name : Hafiz
Enter Patient Number: P0102
Enter Vaccine Type: Sinovac

Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 1
Enter Patient Name : Hani
Enter Patient Number: P0554
Enter Vaccine Type: Pfizer
```

By selecting option 1, users can add new patients in the list by entering patient name, patient number and vaccine type elderly will take. Then, the patient's details will be stored in the system.

Task 2 : Delete Patient Details Who Have Received Vaccine

```
Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 2
Delete Record for Patient Who Have Received Vaccine :
Patient name : John
Patient Number : P0001
```

After elderly have received the vaccine, users can select option 2 to remove the patient's record from the system.

Task 3 : Display All Records

```
Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 3
Patient 1:
Patient name : Hafiz
Patient Number : P0102
Vaccine type to be received : Sinovac

Patient 2:
Patient name : Hani
Patient Number : P0554
Vaccine type to be received : Pfizer
```

Users can select option 3 to display all patient's records in the queue. The system will show every patient name, patient number and vaccine type to be received.

Task 4 : Search Patient Details

```
Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 4
Enter Patient Number : P0554
Patient name : Hani
Patient Number : P0554
Vaccine type to be received : Pfizer
```

```
Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 4
Enter Patient Number : 55
Patient details not found in the queue.
```

Users can select option 4 to search specific patient's details. By entering the patient number, the system will show the respective patient's details if it exists. However, the system will show the message "Patient details not found in the queue." to indicate that either patient number is invalid or not exists in the queue.

Exiting the System

```
====Vaccine Queue System for Elderly====

Menu :
1. Insert New Patient Details
2. Delete Patient Details Who Have Received Vaccine
3. Display All Records
4. Search Patient Details
5. Exit

Enter your choice: 5
Exiting program.
```

By selecting option 5, users will exit the vaccine queue system and remove all patient's records.

Part 4: Task Distribution and Log Sheet

4.1 Task Distribution Table

Task	Assigned Member	Deadline	Status
Write Synopsis, Objective, and System Requirements	Chew Chiu Xian	16/1/2025	Completed
Design Class Diagram	Syarifah Dania binti Syed Abu Bakar	16/1/2025	Completed
Write C++ Code for Queue Operations	Harini A/P Sangaran	16/1/2025	Completed
Test C++ Code	Harini A/P Sangaran, Syarifah Dania binti Syed Abu Bakar	16/1/2025	Completed
Create Presentation Slides	Mohamed Ali Mohamed Ali	12/1/2025	Completed
Prepare Log Sheet	Chew Chiu Xian, Mohamed Ali Mohamed Ali	16/1/2025	Completed

4.2 Group Meeting Log Sheet

Date	Time	Attendance	Task	Details	Status
9/1/2025	11.00 a.m. - 1.00 p.m.	All members present	Project planning	Discuss project scope and finalize task distribution between members	Completed
10/1/2025	11.00 a.m. - 1.00 p.m.	All members present	Discuss the Introduction of the Project	Prepare the introduction of the project by discussing the scope and objective of the project.	Completed
11/1/2025	9.00 - 11.00 p.m.	All members present	Preparing UML Diagram and Use Case Diagram	Discuss between members about UML diagram and Use Case Diagram	Completed
12/1/2025	11.00 a.m. - 12.00 p.m.	All members present	Prototyping System Design	Implement the code by adding core functions of the code which is queue simulation.	Completed
12/1/2025	9.00 - 11.00 p.m.	All members present	System Checking and Update	Checking the system whether it is functioning properly and discussing whether any extra features can be added in the code.	Completed
13/1/2025	9.00 - 10.00 a.m.	All members present	Prepare Slide Presentation	Prepare presentation slide and assign members to present which part of the slide	Completed
15/1/2025	3.00 - 5.00 p.m.	All members present	Record Video for Demo the System	Prepare video to demo the system	Completed

Mini Project System Demo Video Link:

https://youtu.be/S_Uzw3hMZ0s